

Лабораторна робота №1

Тема: Основи розробки на Python, Git та стандарти стилю коду

Мета:

1. Ознайомитися з основами роботи з розподіленою системою контролю версій Git, платформою GitHub та створенням ізольованих віртуальних середовищ (virtualenv).
 2. Навчитись створювати та структурувати проекти на Python для забезпечення багаторазового використання коду (модульність).
 3. Опанувати стандарти оформлення коду PEP-8 та навчитись перевіряти код за допомогою сучасного швидкого статичного аналізатора **Ruff**.
 4. Закріпити навички роботи з умовними операторами, циклами, функціями, колекціями (списки, кортежі, словники, множини), обробкою винятків та збереженням даних (CSV, JSON) у контексті прикладних задач кібербезпеки.
-

План виконання роботи

1. **Робота з Git/GitHub:** Створення репозиторію, налаштування обов'язкового віртуального середовища (virtualenv), налаштування правильної структури проекту.
 2. **Ознайомлення з правилами PEP-8:** Вивчення стандартів написання якісного та читабельного коду.
 3. **Налаштування аналізатора Ruff:** Встановлення та інтеграція сучасного лінера та форматизатора коду.
 4. **Виконання індивідуальних завдань:** Реалізація трьох комплексних завдань відповідно до свого варіанту.
 5. **Аналіз коду та підготовка звіту:** Перевірка написаного коду через Ruff та оформлення фінального звіту.
-

1. Робота з Git та структура репозиторію

Крок 1.1: Створення репозиторію

1. Створіть публічний репозиторій на **GitHub** із назвою, наприклад, cybersecurity-python-labs.
2. При створенні додайте файл .gitignore для Python (він автоматично ігноруватиме папки на кшталт __pycache__ та системні файли).
3. Клонуйте репозиторій на свій локальний комп'ютер.

Крок 1.2: Створення та активація віртуального середовища (virtualenv)

Обов'язковим кроком при розробці проектів на Python є використання ізольованого віртуального середовища. Це дозволяє ізолювати залежності проекту (наприклад, Ruff та інші бібліотеки) від глобальної системи та уникнути конфліктів версій.

1. Перейдіть до папки вашого локального репозиторію в терміналі.
2. Створіть віртуальне середовище за допомогою вбудованого модуля `venv` (рекомендовано для Python 3) або сторонньої утиліти `virtualenv`:

```
python -m venv .venv
```

3. Активуйте створене віртуальне середовище:

- Для macOS/Linux (Bash/Zsh):

```
source .venv/bin/activate
```

- Для Windows (Command Prompt):

```
.venv\Scripts\activate
```

- Для Windows (PowerShell):

```
.\.venv\Scripts\Activate.ps1
```

Після успішної активації у лівій частині рядка запрошення терміналу з'явиться префікс (`.venv`), який свідчить про те, що всі команди `python` та `pip` тепер виконуватимуться в ізольованому середовищі.

4. Переконайтеся, що папка `.venv/` або `venv/` додана до вашого файлу `.gitignore`, щоб тимчасові та службові файли віртуального середовища випадково не потрапили у ваш GitHub-репозиторій.

Крок 1.3: Організація структури репозиторію

Для того, щоб код та налаштування можна було легко перевикористовувати в наступних лабораторних роботах, структура репозиторію має бути наступною:

```
cybersecurity-python-labs/
├── .gitignore
├── requirements.txt
├── shared/
│   ├── __init__.py
│   └── student.py
├── labs/
│   └── lab01/
│       ├── __init__.py
│       ├── main.py
│       ├── task1.py
│       ├── task2.py
│       ├── task3.py
│       └── data/
│           ├── users.csv
│           └── log.json
└── # Ігнорування кешу, звітів Ruff та файлів баз даних
    # Список залежностей проекту (наприклад, ruff)
    # Спільні модулі, що перевикористовуються в усіх ЛР
    # Маркер пакета
    # Модуль з даними студента (ПІБ, група, варіант)
    # Папка для лабораторних робіт
    # Лабораторна робота №1
    # Головний файл для демонстрації роботи
    # Завдання 1: Аналізатор надійності паролів
    # Завдання 2: Система контролю доступу
    # Завдання 3: Хешування, бази даних та логування
    # Тека для збереження результатів (CSV, JSON)
    # База користувачів (створюється динамічно)
    # Журнал подій авторизації (створюється динамічно)
```

Крок 1.4: Створення спільного модуля student.py

У папці shared/ створіть файл student.py. У ньому визначте змінні, які містять ваші персональні дані:

```
# shared/student.py

STUDENT_NAME = "Ваше Прізвище Ім'я По батькові"
GROUP_NAME = "Ваша Група (наприклад, КБ-301)"
VARIANT_NUMBER = 1 # Ваш номер варіанту
```

У подальших лабораторних роботах ви будете імпортувати цей модуль, уникаючи дублювання коду. Для імпорту модуля із суміжної папки можна додати шлях до системного пошуку Python або запускати код як пакет:

```
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
'../../..')))
from shared.student import STUDENT_NAME, GROUP_NAME, VARIANT_NUMBER
```

2. Стандарти стилю коду PEP-8

Код має бути написаний охайно, читабельно та відповідати стандарту PEP-8.

Основні правила PEP-8 (Довідник):

Правило	Правильно	Неправильно
Відступи	4 пробіли (без використання табуляції)	Табуляція або суміш пробілів та табів
Довжина рядка	Не більше 79 символів (72 для docstring/коментарів)	Рядки занадто довгі, що виходять за межі екрана
Порожні рядки	2 порожні рядки перед функціями та класами верхнього рівня, 1 — всередині класу між методами	Відсутність або надмірна кількість порожніх рядків
Імпорти	Кожен імпорт на окремому рядку. Стандартні -> Сторонні -> Локальні	import os, sys на одному рядку або хаотичний порядок
Пробіли	x = 1, y == 2 (пробіли навколо операторів)	x=1, y==2
Пробіли в дужках	good = (1, 2)	bad = (1, 2)
Іменування	Змінні та функції: lower_case_with_under_scores Константи: UPPER_CASE_WITH_UNDER_SCORES Класи: CapitalizedWords	MyVariable, DoTask(), maxsize = 100
Коментарі	# Пояснення (з	#пояснення (без пробілу)

Правило	Правильно	Неправильно
	обов'язковим пробілом після #)	
Docstrings	"""Короткий опис функції або модуля."""	Звичайні коментарі замість докстрінгів
Аргументи за замовчуванням	def func(x=1): (без пробілів навколо =)	def func(x = 1):

3. Перевірка коду за допомогою Ruff

Замість застарілого `pylint` у сучасній розробці використовується **Ruff** — надзвичайно швидкий лінтер та форматизатор коду, написаний на Rust. Він поєднує в собі функції `flake8`, `pylint`, `black`, `isort` та інших інструментів, виконуючи перевірку у 10-100 разів швидше.

Важливо: Всі команди встановлення та запуску Ruff повинні виконуватися виключно у вашому **активованому віртуальному середовищі** (`.venv`)!

Інструкція з використання Ruff:

1. **Встановлення Ruff:** Відкрий термінал у вашому проєкті та виконай:

```
pip install ruff
```

Додайте `ruff` у файл `requirements.txt`.

2. **Запуск лінера (аналіз коду):** Щоб перевірити весь проєкт на наявність помилок стилю, невикористаних імпортів або багів:

```
ruff check .
```

Ruff вкаже на всі порушення PEP-8, невикористані змінні, відсутність docstring та інші проблеми.

3. **Автоматичне виправлення помилок:** Багато стилістичних помилок Ruff може виправити самостійно. Для цього запустіть:

```
ruff check --fix .
```

4. **Автоматичне форматування коду:** Щоб автоматично привести весь код до ідеального вигляду (правильні відступи, переноси рядків, лапки):

```
ruff format .
```

4. Індивідуальні завдання

Кожен студент виконує завдання відповідно до свого **номера варіанта** (імпортується з `shared/student.py`).

Завдання 1: Комплексний аналізатор надійності паролів

Реалізуйте програму, яка оцінює стійкість паролів користувача до компрометації.

Кроки виконання:

1. Імпортуйте ваші персональні дані (ПІБ, варіант) з модуля `shared/student.py`.
 2. Завантажте вихідний список паролів та критерії згідно з вашим варіантом.
 3. Використовуючи стандартний модуль `random`, згенеруйте 3 випадкові індекси зі списку паролів, візьміть відповідні паролі та додайте їх дублікати в кінець початкового списку (таким чином імітується повторне використання паролів).
 4. Оцініть надійність кожного пароля за наступним алгоритмом:
 - **Заборонений:** Якщо пароль входить до списку `forbidden_passwords` або його довжина менша за `min_length` з критеріїв.
 - **Слабкий:** Якщо пароль не є забороненим і виконує хоча б один з критеріїв безпеки (містить хоча б одну цифру, велику літеру, спеціальний символ або малу літеру).
 - **Середній:** Якщо пароль відповідає мінімальній довжині та деяким (але не всім) критеріям з різних груп символів.
 - **Сильний:** Якщо пароль відповідає абсолютно всім критеріям безпеки (довжина, цифра, велика літера, спеціальний символ), але його довжина менша за `min_length + 4` символів.
 - **Дуже сильний:** Якщо пароль відповідає всім критеріям безпеки, має довжину `min_length + 4` або більше символів, та є абсолютно унікальним у всьому списку паролів.
 5. Виведіть результат аналізу на екран у зручному табличному форматі.
-

Завдання 2: Багаторівнева система контролю доступу

Розробіть прототип системи розмежування доступу користувачів до конфіденційних ресурсів.

Кроки виконання:

1. Імпортуйте вхідні дані для свого варіанту (словник користувачів `users`, список ресурсів `resources`, рівні безпеки `security_levels`, множину заблокованих користувачів `blocked_users`).
2. Виведіть список усіх ресурсів системи на екран, де числовий рівень безпеки ресурсу (від 1 до 4) замінено його текстовою назвою з кортежу `security_levels` (наприклад, замість 4 вивести `Secret` або `Top Secret`).
3. Реалізуйте алгоритм перевірки доступу кожного користувача до кожного ресурсу.
 - **Алгоритм перевірки:**
 - Якщо користувача немає в системі (словнику `users`) -> **DENY (User**

not found).

- Якщо користувач є у списку заблокованих `blocked_users` -> **DENY (User is blocked).**
- Якщо обліковий запис неактивний (`active == False`) -> **DENY (Account inactive).**
- Якщо числовий рівень допуску користувача (`clearance`) більший або дорівнює рівню безпеки ресурсу -> **ALLOW.**
- Якщо рівень допуску менший за рівень ресурсу -> **DENY (Insufficient clearance).**

4. Виведіть результати перевірки на екран у форматі: `user=[ім'я_користувача]`
`resource=[назва_ресурсу] -> ALLOW / DENY ([причина_відмови])`

Завдання 3: Безпечне хешування, CSV-база та JSON-логування з винятками

Реалізуйте модуль для безпечної реєстрації та автентифікації користувачів, використовуючи функції, декоратори та роботу з файлами.

Кроки виконання:

1. **Хешування:** Створіть функцію `generate_hash(password: str, salt: str = "00000") -> str`.
 - Функція має повертати шістнадцятковий хеш від конкатенації пароля та солі за допомогою алгоритму хешування з вашого варіанту.
 - Якщо пароль або сіль порожні (`None` або `""`) — згенеруйте виняток `ValueError`.
 - Якщо пароль коротший за мінімальну довжину з варіанту — згенеруйте власний виняток `ValidationError` (успадкований від `Exception`).
2. **Персональна сіль:** Створіть сіль як рядок із 5 символів, що містить ваш варіант із `shared/student.py`, доповнений зліва нулями (наприклад, `"00005"` для 5 варіанту).
3. **Реєстрація користувачів:**
 - Створіть кортеж `users_to_register`, що містить 10 записів (користувачів) у форматі (логін, пароль).
 - Напишіть функцію `create_user(username, password)`. Вона викликає `generate_hash` з вашою персональною сіллю та повертає кортеж (`username, hash_value`).
 - Напишіть функцію `create_users(users_list)` для створення бази даних. Вона обробляє весь список користувачів та записує їх у файл `labs/lab01/data/users.csv` у форматі логін, хеш_пароля.

Переконайтеся, що папка `data` створюється автоматично.

4. **Читання бази даних:** Зчитайте вміст CSV-файлу у список `users_db` та виведіть його у вигляді структурованої таблиці на екран.
5. **Автентифікація:** Напишіть функцію `login(username: str, password: str) -> bool`.
 - Перевірте, чи існує користувач у `users_db` та чи збігається хеш введеного пароля (з урахуванням солі) з тим, що збережений у CSV.
 - Якщо логін або пароль порожні — згенеруйте `ValueError`.
6. **Логування подій (Декоратор):** Створіть декоратор `@log_event`, який записує кожну спробу входу у файл `labs/lab01/data/log.json` у форматі:

```
{
  "event": "login",
  "user": "ім'я_користувача",
  "result": "success" или "failure",
  "timestamp": "YYYY-MM-DD HH:MM:SS",
  "args": [],
  "kwargs": {}
}
```

7. **Обробка винятків:** Увесь код роботи з файлами та входу обгорніть у блоки `try...except` із обробкою винятків: `FileNotFoundError`, `PermissionError`, `IOError`, `ValidationError` та `ValueError`.
8. Організуйте весь запуск через головну функцію `main()`.

УВАГА! Після виконання всіх завдань, код має бути розміщений у GitHub репозиторії!

5. Вхідні дані для варіантів (Завдання 1 та 2)

Використовуйте дані свого варіанту (від 1 до 15):

Варіант 1

Завдання 1:

```
passwords = ["password123", "Qwerty!2023", "admin", "MyP@ssw0rd", "123456",
"SecurePass!", "test", "P@ssw0rd123", "welcome", "StrongP@ss1"]
criteria = {"min_length": 8, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"password", "123456", "admin", "test", "welcome",
"qwerty"}
```

Завдання 2:

```
users = {
  "admin001": {"role": "administrator", "clearance": 4, "department": "IT",
"active": True},
  "user123": {"role": "analyst", "clearance": 2, "department": "Security",
"active": True},
  "guest789": {"role": "guest", "clearance": 1, "department": "External",
```

```

"active": True},
  "manager456": {"role": "manager", "clearance": 3, "department":
"Operations", "active": True},
  "contractor99": {"role": "contractor", "clearance": 1, "department":
"External", "active": False}
}
resources = [("database_backup", 4), ("user_logs", 2), ("public_docs", 1),
("financial_reports", 3), ("system_config", 4), ("training_materials", 1),
("security_policies", 3), ("audit_logs", 4), ("employee_data", 3),
("temp_files", 1)]
security_levels = ("Public", "Internal", "Confidential", "Secret")
blocked_users = {"contractor99", "temp_user", "suspended_acc"}

```

Варіант 2

Завдання 1:

```

passwords = ["Hello123!", "simple", "CompL3x@Pass", "password", "Str0ng#2023",
"weak", "MySecur3!", "12345", "Advanced@1", "basic"]
criteria = {"min_length": 10, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"password", "simple", "weak", "basic", "12345", "hello"}

```

Завдання 2:

```

users = {
  "sysadmin02": {"role": "system_admin", "clearance": 4, "department":
"Infrastructure", "active": True},
  "analyst234": {"role": "security_analyst", "clearance": 3, "department":
"SOC", "active": True},
  "developer567": {"role": "developer", "clearance": 2, "department":
"Development", "active": True},
  "intern890": {"role": "intern", "clearance": 1, "department": "HR",
"active": True},
  "external123": {"role": "external", "clearance": 1, "department": "Vendor",
"active": False}
}
resources = [("prod_database", 4), ("dev_environment", 2), ("documentation", 1),
("source_code", 3), ("server_configs", 4), ("test_data", 2), ("compliance_docs",
3), ("system_logs", 4), ("project_files", 2), ("public_wiki", 1)]
security_levels = ("Open", "Internal", "Restricted", "Top Secret")
blocked_users = {"external123", "old_account", "test_user"}

```

Варіант 3

Завдання 1:

```

passwords = ["UserPass1!", "temp", "Cyber$ecur1ty", "guest", "P0w3rful@Pass",
"login", "Defens3#2023", "abc123", "Elit3@Secur", "demo"]
criteria = {"min_length": 9, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"temp", "guest", "login", "demo", "abc123", "user"}

```

Завдання 2:

```

users = {
  "security_chief": {"role": "security_officer", "clearance": 4, "department":
"Security", "active": True},
  "network_admin": {"role": "network_admin", "clearance": 3, "department":
"Network", "active": True},
  "help_desk": {"role": "support", "clearance": 1, "department": "Support",
"active": True},
  "auditor_ext": {"role": "auditor", "clearance": 3, "department": "Audit",

```



```

"active": True},
  "temp_worker": {"role": "temporary", "clearance": 1, "department": "Temp",
"active": False}
}
resources = [("incident_reports", 4), ("network_topology", 3), ("user_manual",
1), ("vulnerability_scans", 3), ("root_access", 4), ("help_tickets", 1),
("penetration_tests", 4), ("firewall_rules", 3), ("software_licenses", 2),
("faq_docs", 1)]
security_levels = ("Unrestricted", "Limited", "Sensitive", "Classified")
blocked_users = {"temp_worker", "fired_employee", "compromised_acc"}

```

Варіант 4

Завдання 1:

```

passwords = ["Security@2023", "pass", "MyStr0ng#Key", "root", "Advanc3d@Pass",
"user", "Protec7!Pass", "1234", "Elite@Secur1ty", "admin123"]
criteria = {"min_length": 7, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"pass", "root", "user", "1234", "admin123", "password"}

```

Завдання 2:

```

users = {
  "ciso_office": {"role": "ciso", "clearance": 4, "department": "Executive",
"active": True},
  "threat_hunter": {"role": "threat_analyst", "clearance": 3, "department":
"Threat Intel", "active": True},
  "junior_dev": {"role": "junior_developer", "clearance": 2, "department":
"Development", "active": True},
  "visitor_acc": {"role": "visitor", "clearance": 1, "department": "Guest",
"active": True},
  "legacy_sys": {"role": "legacy", "clearance": 2, "department": "Legacy",
"active": False}
}
resources = [("threat_intelligence", 4), ("malware_samples", 3),
("coding_guidelines", 2), ("visitor_wifi", 1), ("strategic_plans", 4),
("demo_environment", 1), ("risk_assessments", 3), ("crypto_keys", 4),
("api_documentation", 2), ("guest_portal", 1)]
security_levels = ("Guest", "Employee", "Privileged", "Executive")
blocked_users = {"legacy_sys", "malicious_user", "expired_guest"}

```

Варіант 5

Завдання 1:

```

passwords = ["DataS3cur3!", "123", "Crypt0@Analysis", "test123", "Quantum#2023",
"access", "Secur1ty@Pro", "password1", "Adv@nced123", "guest123"]
criteria = {"min_length": 12, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"123", "test123", "access", "password1", "guest123",
"admin"}

```

Завдання 2:

```

users = {
  "forensic_lead": {"role": "forensic_analyst", "clearance": 4, "department":
"Forensics", "active": True},
  "compliance_off": {"role": "compliance_officer", "clearance": 3,
"department": "Compliance", "active": True},
  "trainee_sec": {"role": "trainee", "clearance": 1, "department": "Training",
"active": True},
  "vendor_tech": {"role": "vendor_support", "clearance": 2, "department":

```

```

"Vendor", "active": True},
  "archived_usr": {"role": "archived", "clearance": 1, "department":
"Archive", "active": False}
}
resources = [("forensic_images", 4), ("compliance_reports", 3),
("training_videos", 1), ("vendor_tools", 2), ("evidence_locker", 4),
("certification_docs", 1), ("audit_findings", 3), ("chain_of_custody", 4),
("support_tickets", 2), ("learning_modules", 1)]
security_levels = ("Basic", "Standard", "Protected", "Maximum")
blocked_users = {"archived_usr", "terminated_vendor", "security_breach"}

```

Варіант 6

Завдання 1:

```

passwords = ["InfoS3c@2023", "simple123", "Def3ns3@Key", "public",
"Encrypt3d#Pass", "basic123", "Secur3@Analysis", "temp123", "Pr0t3ct@Data",
"default"]
criteria = {"min_length": 8, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"simple123", "public", "basic123", "temp123", "default",
"guest"}

```

Завдання 2:

```

users = {
  "red_team_lead": {"role": "red_team", "clearance": 4, "department": "Red
Team", "active": True},
  "blue_team_analyst": {"role": "blue_team", "clearance": 3, "department":
"Blue Team", "active": True},
  "purple_team_coord": {"role": "purple_team", "clearance": 3, "department":
"Purple Team", "active": True},
  "student_intern": {"role": "student", "clearance": 1, "department":
"Academia", "active": True},
  "retired_expert": {"role": "retired", "clearance": 2, "department":
"Emeritus", "active": False}
}
resources = [("attack_scenarios", 4), ("defense_playbooks", 3),
("exercise_plans", 3), ("research_papers", 1), ("exploit_tools", 4),
("student_resources", 1), ("simulation_results", 3), ("red_team_tools", 4),
("blue_team_reports", 3), ("public_research", 1)]
security_levels = ("Academic", "Operational", "Tactical", "Strategic")
blocked_users = {"retired_expert", "academic_violator", "leaked_account"}

```

Варіант 7

Завдання 1:

```

passwords = ["NetworkS3c!", "easy", "Firewa11@Pass", "anonymous",
"Intrus10n#Detect", "sample", "Malwar3@Scan", "qwerty", "Vulnerab1l!ty",
"common"]
criteria = {"min_length": 9, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"easy", "anonymous", "sample", "qwerty", "common",
"password"}

```

Завдання 2:

```

users = {
  "incident_commander": {"role": "incident_response", "clearance": 4,
"department": "CSIRT", "active": True},
  "malware_analyst": {"role": "malware_researcher", "clearance": 3,
"department": "Research", "active": True},

```

```

    "monitoring_tech": {"role": "monitoring", "clearance": 2, "department":
"NOC", "active": True},
    "customer_rep": {"role": "customer_service", "clearance": 1, "department":
"Customer", "active": True},
    "backup_service": {"role": "service_account", "clearance": 2, "department":
"System", "active": False}
}
resources = [("incident_playbook", 4), ("malware_lab", 3),
("monitoring_dashboards", 2), ("customer_portal", 1), ("emergency_procedures",
4), ("service_desk", 1), ("reverse_engineering", 3), ("alert_systems", 2),
("escalation_matrix", 3), ("knowledge_base", 1)]
security_levels = ("Public Access", "Authorized", "Privileged", "Critical")
blocked_users = {"backup_service", "deactivated_svc", "policy_violation"}

```

Варіант 8

Завдання 1:

```

passwords = ["ThreatH@nt3r", "weak123", "P3n3trat10n@Test", "visitor",
"Cyber@Defense2023", "normal", "Incident@R3sp0nse", "standard", "Risk@Analys1s",
"typical"]
criteria = {"min_length": 10, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"weak123", "visitor", "normal", "standard", "typical",
"admin"}

```

Завдання 2:

```

users = {
    "crypto_specialist": {"role": "cryptographer", "clearance": 4, "department":
"Cryptography", "active": True},
    "privacy_officer": {"role": "privacy_analyst", "clearance": 3, "department":
"Privacy", "active": True},
    "data_scientist": {"role": "data_analyst", "clearance": 2, "department":
"Analytics", "active": True},
    "field_engineer": {"role": "field_support", "clearance": 2, "department":
"Field Ops", "active": True},
    "test_account": {"role": "testing", "clearance": 1, "department": "QA",
"active": False}
}
resources = [("encryption_keys", 4), ("privacy_policies", 3),
("anonymized_data", 2), ("field_reports", 2), ("crypto_algorithms", 4),
("consent_forms", 1), ("data_classification", 3), ("key_management", 4),
("statistical_models", 2), ("public_datasets", 1)]
security_levels = ("Unclassified", "For Official Use", "Confidential", "Secret")
blocked_users = {"test_account", "gdpr_violation", "data_breach_user"}

```

Варіант 9

Завдання 1:

```

passwords = ["Digital@F0r3nsics", "plain", "Encrypt10n@Key", "member",
"Security@Audit2023", "regular", "Hack3r@D3fense", "ordinary", "Threat@Intel",
"usual"]
criteria = {"min_length": 8, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"plain", "member", "regular", "ordinary", "usual",
"user"}

```

Завдання 2:

```

users = {
    "cloud_architect": {"role": "cloud_security", "clearance": 4, "department":

```

```

"Cloud", "active": True},
  "devops_engineer": {"role": "devops", "clearance": 3, "department":
"DevOps", "active": True},
  "qa_tester": {"role": "quality_assurance", "clearance": 2, "department":
"QA", "active": True},
  "partner_access": {"role": "partner", "clearance": 2, "department":
"Partnership", "active": True},
  "migrated_user": {"role": "migrated", "clearance": 1, "department":
"Migration", "active": False}
}
resources = [("cloud_configs", 4), ("deployment_pipelines", 3),
("test_environments", 2), ("partner_apis", 2), ("infrastructure_code", 4),
("shared_resources", 1), ("container_registry", 3), ("secrets_vault", 4),
("build_artifacts", 2), ("public_endpoints", 1)]
security_levels = ("Development", "Staging", "Production", "Critical
Infrastructure")
blocked_users = {"migrated_user", "container_breach", "pipeline_compromise"}

```

Варіант 10

Завдання 1:

```

passwords = ["S0cial@Engineer", "basic", "Phish1ng@D3tect", "client",
"Ransomwar3@Protect", "general", "Zero@D4y", "generic", "Bug@B0untY",
"standard123"]
criteria = {"min_length": 11, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"basic", "client", "general", "generic", "standard123",
"guest"}

```

Завдання 2:

```

users = {
  "iot_specialist": {"role": "iot_security", "clearance": 3, "department":
"IoT", "active": True},
  "mobile_analyst": {"role": "mobile_security", "clearance": 3, "department":
"Mobile", "active": True},
  "web_developer": {"role": "web_developer", "clearance": 2, "department":
"Web", "active": True},
  "api_consumer": {"role": "api_user", "clearance": 2, "department":
"Integration", "active": True},
  "demo_account": {"role": "demonstration", "clearance": 1, "department":
"Demo", "active": False}
}
resources = [("iot_firmware", 3), ("mobile_policies", 3), ("web_applications",
2), ("api_gateway", 2), ("device_certificates", 3), ("app_store", 1),
("vulnerability_database", 3), ("device_management", 3), ("integration_docs",
2), ("demo_content", 1)]
security_levels = ("Consumer", "Business", "Enterprise", "Critical Systems")
blocked_users = {"demo_account", "compromised_device", "malicious_app"}

```

Варіант 11

Завдання 1:

```

passwords = ["APT@Detect10n", "simple", "Red@Team2023", "participant",
"Blue@T3am", "common123", "Purple@T34m", "regular123", "Gr33n@Team",
"normal123"]
criteria = {"min_length": 7, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"simple", "participant", "common123", "regular123",
"normal123", "test"}

```

Завдання 2:

```
users = {
    "risk_manager": {"role": "risk_analyst", "clearance": 4, "department": "Risk
Management", "active": True},
    "business_analyst": {"role": "business_analyst", "clearance": 2,
"department": "Business", "active": True},
    "legal_counsel": {"role": "legal", "clearance": 3, "department": "Legal",
"active": True},
    "contractor_dev": {"role": "contractor", "clearance": 2, "department":
"Contract", "active": True},
    "obsolete_system": {"role": "legacy_system", "clearance": 1, "department":
"Legacy", "active": False}
}
resources = [("risk_registers", 4), ("business_requirements", 2),
("legal_documents", 3), ("contract_code", 2), ("governance_framework", 4),
("meeting_minutes", 1), ("regulatory_reports", 3), ("executive_dashboards", 4),
("project_specs", 2), ("public_statements", 1)]
security_levels = ("Public", "Internal Use", "Restricted", "Highly Restricted")
blocked_users = {"obsolete_system", "contract_expired", "legal_hold"}
```

Варіант 12

Завдання 1:

```
passwords = ["SIEM@An4lysis", "easy123", "SOC@Analyst", "observer",
"Threat@Hunting", "viewer", "Incid3nt@Handle", "monitor", "Log@An4lysis",
"watcher"]
criteria = {"min_length": 9, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"easy123", "observer", "viewer", "monitor", "watcher",
"admin"}
```

Завдання 2:

```
users = {
    "devsecops_lead": {"role": "devsecops", "clearance": 4, "department":
"DevSecOps", "active": True},
    "security_engineer": {"role": "security_engineer", "clearance": 3,
"department": "Security Engineering", "active": True},
    "automation_tech": {"role": "automation", "clearance": 2, "department":
"Automation", "active": True},
    "api_developer": {"role": "api_developer", "clearance": 2, "department":
"API", "active": True},
    "sandbox_env": {"role": "sandbox", "clearance": 1, "department": "Testing",
"active": False}
}
resources = [("security_pipelines", 4), ("secure_coding_standards", 3),
("automation_scripts", 2), ("api_specifications", 2), ("threat_models", 4),
("testing_frameworks", 1), ("security_gates", 3), ("vulnerability_scans", 4),
("integration_tests", 2), ("mock_services", 1)]
security_levels = ("Sandbox", "Development", "Secure", "Production Critical")
blocked_users = {"sandbox_env", "pipeline_breach", "automation_fail"}
```

Варіант 13

Завдання 1:

```
passwords = ["Compli4nc3@Check", "weak", "Risk@Ass3ssment", "guest",
"Vulner4bility@Scan", "temp", "P3netration@Test", "demo", "S3curity@Audit",
"trial"]
criteria = {"min_length": 8, "require_digits": True, "require_upper": True,
"require_special": True}
```

```
forbidden_passwords = {"weak", "guest", "temp", "demo", "trial", "password"}
```

Завдання 2:

```
users = {
    "ai_security_expert": {"role": "ai_security", "clearance": 4, "department":
    "AI Security", "active": True},
    "ml_engineer": {"role": "ml_engineer", "clearance": 3, "department":
    "Machine Learning", "active": True},
    "data_engineer": {"role": "data_engineer", "clearance": 2, "department":
    "Data Engineering", "active": True},
    "research_assistant": {"role": "researcher", "clearance": 2, "department":
    "Research", "active": True},
    "training_bot": {"role": "bot_account", "clearance": 1, "department":
    "Automation", "active": False}
}
resources = [("ai_models", 4), ("training_datasets", 3), ("data_pipelines", 2),
("research_notebooks", 2), ("model_artifacts", 4), ("synthetic_data", 1),
("adversarial_tests", 3), ("model_registry", 4), ("feature_stores", 2),
("public_models", 1)]
security_levels = ("Open Source", "Internal Research", "Proprietary", "Trade
Secret")
blocked_users = {"training_bot", "model_theft", "data_poisoning_acc"}
```

Варіант 14

Завдання 1:

```
passwords = ["C2@Command", "plain123", "Backdoor@D3tect", "public123",
"Rootkit@Hunt", "access123", "Exploit@An4lysis", "basic", "Payload@D3code",
"default123"]
criteria = {"min_length": 10, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"plain123", "public123", "access123", "basic",
"default123", "user"}
```

Завдання 2:

```
users = {
    "blockchain_dev": {"role": "blockchain_developer", "clearance": 3,
    "department": "Blockchain", "active": True},
    "smart_contract_auditor": {"role": "contract_auditor", "clearance": 3,
    "department": "Audit", "active": True},
    "crypto_trader": {"role": "trader", "clearance": 2, "department": "Trading",
    "active": True},
    "wallet_user": {"role": "wallet_user", "clearance": 1, "department":
    "Users", "active": True},
    "mining_pool": {"role": "miner", "clearance": 1, "department": "Mining",
    "active": False}
}
resources = [("smart_contracts", 3), ("audit_reports", 3),
("trading_algorithms", 2), ("wallet_interface", 1), ("private_keys", 3),
("public_blockchain", 1), ("defi_protocols", 3), ("validator_nodes", 3),
("market_data", 2), ("community_forum", 1)]
security_levels = ("Public Blockchain", "Permissioned", "Private Network",
"Institutional")
blocked_users = {"mining_pool", "flash_loan_attack", "rug_pull_scam"}
```

Варіант 15

Завдання 1:

```
passwords = ["IoT@S3curity", "standard", "Blockchain@Pr0tect", "typical123",
```

```
"AI@Cybersec", "normal", "Quantum@Crypt0", "general123", "Edge@S3curity",
"common"]
criteria = {"min_length": 8, "require_digits": True, "require_upper": True,
"require_special": True}
forbidden_passwords = {"standard", "typical123", "normal", "general123",
"common", "guest"}
```

Завдання 2:

```
users = {
    "quantum_researcher": {"role": "quantum_security", "clearance": 4,
"department": "Quantum Research", "active": True},
    "post_quantum_dev": {"role": "pq_cryptographer", "clearance": 4,
"department": "Post-Quantum", "active": True},
    "network_security": {"role": "network_security", "clearance": 3,
"department": "Network Security", "active": True},
    "crypto_intern": {"role": "crypto_intern", "clearance": 1, "department":
"Internship", "active": True},
    "quantum_sim": {"role": "simulator", "clearance": 2, "department":
"Simulation", "active": False}
}
resources = [("quantum_algorithms", 4), ("pq_implementations", 4),
("network_protocols", 3), ("learning_materials", 1), ("quantum_keys", 4),
("educational_content", 1), ("hybrid_systems", 3), ("quantum_computers", 4),
("crypto_libraries", 2), ("tutorials", 1)]
security_levels = ("Educational", "Research", "Classified Research", "Quantum
Secure")
blocked_users = {"quantum_sim", "quantum_attack", "algorithm_theft"}
--
```

6. Таблиця варіантів (Завдання 3)

Варіант	Алгоритм хешування	Мінімальна довжина пароля
1	sha3_512	12
2	sha3_224	10
3	sha1	8
4	sha512	14
5	sha3_256	16
6	blake2s	9
7	sha384	15
8	sha256	11
9	sha224	13
10	sha3_384	10
11	blake2b	12
12	md5	8
13	sha3_256	14
14	sha512	16
15	sha1	9

7. Контрольні питання для обговорення (Захист роботи)

1. **Про ізоляцію залежностей (virtualenv):** Чому професійні розробники наполягають на використанні віртуальних середовищ навіть для найменших проектів? Що може піти не так у вашій операційній системі, якщо ви інстальватимете всі пакети (на кшталт Ruff чи Flask) глобально через `pip install`?
2. **Про гігієну репозиторію та .gitignore:** Уявіть, що ви випадково завантажили папку `.venv/` або файл `users.csv` із реальними паролями на GitHub. Чому це вважається критичним інцидентом безпеки?
3. **Списки (list), Кортежі (tuple) та Множини (set):** У роботі ви використовували різні типи колекцій: список паролів `passwords`, кортеж рівнів безпеки `security_levels` та множину заблокованих користувачів `blocked_users`. Чому для заблокованих користувачів використовується саме множина (`set`)? Яка різниця у швидкості пошуку елемента (`x in collection`) між списком та множиною, якщо база користувачів зростає до мільйона записів? Чому `security_levels` логічно зробити саме незмінним кортежем (`tuple`)?
4. **Словники (dict):** У Завданні 2 інформація про користувачів представлена у вигляді словника словників (`users`), де ключем є логін, а значенням — інший словник з парами `key: value`. Які переваги дає така структура в порівнянні зі списком кортежів чи списком списків? Як отримати значення департаменту конкретного користувача, не використовуючи цикл, і що станеться, якщо спробувати звернутися до неіснуючого ключа безпосередньо через `users[key]` проти безпечного методу `.get()`?
5. **Про автоматизацію якості коду (Ruff проти ручного контролю):** Якщо ваш код працює абсолютно коректно та виконує всі завдання, чому лінер Ruff все одно може поставити ему низьку оцінку або видати десятки попереджень? Чи може «погано відформатований, але робочий» код приховувати реальні вразливості безпеки?
6. **Про унікальність та повторне використання паролів (Завдання 1):** Під час оцінки надійності ви штучно дублювали паролі. Чому повторне використання одного й того самого пароля (навіть надзвичайно складного) на різних ресурсах є однією з головних проблем сучасної кібербезпеки?
7. **Про незворотність: Хешування проти Шифрування (Завдання 3):** Чому для збереження паролів у файлі `users.csv` ми використовуємо саме одностороннє хешування, а не симетричне шифрування (наприклад, AES)? У чому різниця між цими двома концепціями з точки зору зловмисника, який викрав базу даних?
8. **Про криптографічну сіль (salt):** Яку роль відіграє сіль у безпечному збереженні паролів? Як саме використання унікальної солі захищає базу користувачів від атак із використанням **веселкових таблиць (Rainbow Tables)** та чому однакові паролі двох різних користувачів матимуть абсолютно різні хеші?
9. **Про переваги використання декораторів (Декоратор @log_event):** Чому логування спроб авторизації було винесено саме в декоратор, а не реалізовано всередині самої функції `login()`? Які архітектурні переваги це дає під час

масштабування та аудиту системи?

10. **Про безпечну обробку помилок (Fail-Secure):** Чому під час роботи з файлами та валідацією важливо перехоплювати конкретні винятки (як-от `FileNotFoundError` чи `ValidationError`), а не писати загальний блок `except Exception`? Як детальний вивід системного трасування помилки (`Traceback`) на екран користувача може допомогти зловмиснику?
 11. **Про повторне використання коду та модуль `student.py`:** Ви винесли свої персональні дані в окремий модуль `shared/student.py`. Чому модульний підхід та принцип **DRY (Don't Repeat Yourself)** є фундаментальними не лише для зручності розробника, а й для безпеки програмного забезпечення загалом?
-

Корисні посилання (Теоретичні джерела)

- **PEP-8** <https://peps.python.org/pep-0008/>
- **Списки** https://w3schoolsua.github.io/python/python_lists.html
- **Кортежі** https://w3schoolsua.github.io/python/python_tuples.html
- **Набори (сети)** https://w3schoolsua.github.io/python/python_sets.html
- **Словники** https://w3schoolsua.github.io/python/python_dictionaries.html
- **Умовні конструкції If ... Else**
https://w3schoolsua.github.io/python/python_conditions.html
- **Цикли For** https://w3schoolsua.github.io/python/python_for_loops.html
- **Цикли While** https://w3schoolsua.github.io/python/python_while_loops.html
- **Функції** https://w3schoolsua.github.io/python/python_functions.html
- **Винятки** https://w3schoolsua.github.io/python/python_try_except.html
- **Модулі** https://w3schoolsua.github.io/python/python_modules.html
- **Модуль CSV** <https://docs.python.org/uk/3/library/csv.html>
- **Модуль JSON** <https://docs.python.org/uk/3/library/json.html>
- **Модуль hashlib** <https://docs.python.org/uk/3.10/library/hashlib.html>
- **Робота з файлами** https://w3schoolsua.github.io/python/python_file_handling.html
- **Читання файлів** https://w3schoolsua.github.io/python/python_file_open.html
- **Запис файлу** https://w3schoolsua.github.io/python/python_file_write.html
- **Видалення файлів** https://w3schoolsua.github.io/python/python_file_remove.html
- **Читання та запис файлів** <https://docs.python.org/uk/3/tutorial/inputoutput.html#reading-and-writing-files>
- **Вбудовані винятки Python** <https://docs.python.org/uk/3.13/library/exceptions.html>